

StockPulse

Real-Time Stock News Aggregation Platform

Complete Technical Documentation

Version 1.0

February 2026

Table of Contents

1. What is StockPulse?	3
2. How the System Works	4
2.1 Main Parts of the System	
2.2 Technologies Used	
2.3 News Sources	
3. Features List	7
4. Database Tables	9
5. All API Endpoints	13
6. How Things Work (Flowcharts)	17
7. Step-by-Step Data Flow	21
8. Setup Requirements	24
9. Project Files Explained	26
10. What Can Be Added Later	28

1. What is StockPulse?

StockPulse is a web application that collects and shows stock market news from multiple sources in one place. Instead of visiting 10 different websites to check news about your stocks, you can see everything here.

Main Features

- Real News: Gets actual news articles from Google News RSS feed - not fake or sample data
- Sentiment Tags: Each news article is marked as Positive, Negative, or Neutral based on keywords in the headline
- Watchlist: Save your favorite stocks and quickly access them anytime
- Auto-Scheduler: Set up automatic news fetching - runs in background even when you are not using the app
- Time Filters: See news from last 24 hours, 48 hours, 72 hours, or 7 days
- Sentiment Stats: See count of how many positive, negative, and neutral articles were found
- 167 Stocks: Pre-loaded database of Indian (NSE/BSE) and US (NASDAQ/NYSE) stocks
- Bulk Import: Upload a .txt or .csv file to add many stocks at once
- Modern UI: Beautiful glassmorphism design with smooth animations

Who Is This For?

Anyone who wants to track stock market news quickly - traders, investors, or anyone interested in the stock market. It saves time by bringing all news to one place.

2. How the System Works

2.1 Main Parts of the System

The application has 4 main parts that work together:

Part	What It Does	Technology
Frontend	What user sees - search box, news cards, buttons, watchlist panel	HTML + CSS (Tailwind) + JavaScript
Backend API	Handles all requests - fetching news, saving data, running schedulers	Python Flask (runs on port 5000 or 8888)
Database	Stores everything - news articles, watchlist, schedulers, history	MySQL 8.0 with connection pooling
News Source	Where news comes from - Google aggregates from many news sites	Google News RSS Feed

DIAGRAM: System Architecture

Frontend → Flask API → MySQL Database ↔ Google News RSS

2.2 Technologies Used

Here is the complete list of all technologies and their versions:

Technology	Purpose	Version
Flask	Python web framework for API	3.1.0
MySQL	Database to store all data	8.0 or higher
mysql-connector-python	Connects Python to MySQL	9.1.0
feedparser	Reads RSS feeds from Google News	6.0.11
requests	Makes HTTP requests	2.32.3

APScheduler	Runs background jobs automatically	3.10.4
Tailwind CSS	CSS framework for styling	CDN (Latest)
Font Awesome	Icons used in UI	6.4.0

2.3 News Sources

News comes from Google News RSS which collects articles from these major sources:

International	Indian	Finance Portals
Reuters	Economic Times	Yahoo Finance
Bloomberg	Moneycontrol	Investing.com
CNBC	LiveMint	MarketWatch
Wall Street Journal	Business Standard	Seeking Alpha
Forbes	NDTV Profit	Financial Express
BBC Business	Zee Business	The Hindu Business Line

The system recognizes 25+ news sources and shows their icon and color in the news cards.

3. Features List

Complete list of all features in StockPulse:

ID	Feature	What It Does	Status
F-01	Stock Search	Type stock name or symbol and get suggestions (autocomplete). Works with 167 pre-loaded stocks.	Done
F-02	Live News Fetch	Click 'Fetch News' to get real articles from Google News RSS. Shows title, source, time, sentiment.	Done
F-03	Sentiment Analysis	Each article is tagged Positive/Negative/Neutral using keyword matching in headline.	Done
F-04	Sentiment Stats	Shows count of positive, negative, neutral articles at the top after fetching.	Done
F-05	Time Filters	Filter news by 24 hours, 48 hours, 72 hours, or 7 days old.	Done
F-06	Watchlist	Add stocks to your personal list. Click star icon to add, click again to remove.	Done
F-07	Bulk Import	Upload .txt or .csv file with stock symbols to add many at once to watchlist.	Done
F-08	Scheduler	Create a job to auto-fetch news for all watchlist stocks at a specific time.	Done
F-09	Scheduler History	See all past scheduler runs with total stocks processed and articles found per stock.	Done
F-10	Search History	View all your past searches with timestamps and article counts.	Done
F-11	Source Filter	After fetching, filter news to show only from a specific source.	Done

F-12	Sort Options	Sort news by newest first, oldest first, or by source name.	Done
F-13	Relevance Filter	Automatically removes articles that are not actually about the searched stock.	Done
F-14	URL Deduplication	Same article is not stored twice. Uses MD5 hash of URL to detect duplicates.	Done

4. Database Tables

StockPulse uses MySQL database with 5 tables. Tables are auto-created when the app starts (no need to run SQL manually).

DIAGRAM: Database ER Diagram

watchlist ↔ news ↔ schedulers ↔ scheduler_history, search_history

4.1 watchlist table

Stores user's saved stocks for quick access and scheduler fetching.

Column	Type	Rules	What It Stores
id	INT	Primary Key, Auto Increment	Unique ID for each row
symbol	VARCHAR(50)	NOT NULL, UNIQUE	Stock symbol like TCS, RELIANCE
company_name	VARCHAR(200)	NOT NULL	Full company name
exchange	VARCHAR(20)	Default: NSE	NSE, BSE, NASDAQ, NYSE
added_at	TIMESTAMP	Default: Current Time	When stock was added

4.2 news table

Main table - stores all fetched news articles with sentiment and source info.

Column	Type	Rules	What It Stores
id	INT	Primary Key, Auto Increment	Unique ID
symbol	VARCHAR(50)	NOT NULL, Has Index	Which stock this news is for
title	VARCHAR(500)	NOT NULL	News headline
source	VARCHAR(200)	-	Reuters, CNBC, etc.
url	TEXT	-	Link to full article
url_hash	VARCHAR(32)	UNIQUE	MD5 hash to prevent duplicates
published_at	DATETIME	Has Index	When article was published
snippet	TEXT	-	Short summary of article
sentiment	VARCHAR(20)	Default: neutral	positive, negative, or neutral
time_filter	VARCHAR(10)	Default: 24h	Which time filter was used
fetched_at	TIMESTAMP	Default: Current Time	When we fetched this article
is_scheduler_fetch	BOOLEAN	Default: FALSE	TRUE if fetched by scheduler
scheduler_id	INT	Foreign Key, Has Index	Which scheduler fetched this

4.3 schedulers table

Stores scheduler jobs that auto-fetch news at specific times.

Column	Type	Rules	What It Stores
id	INT	Primary Key, Auto Increment	Unique ID
name	VARCHAR(200)	NOT NULL	Name like 'Morning Fetch'
schedule_time	DATETIME	NOT NULL	When to run the job
time_filter	VARCHAR(20)	Default: 24h	24h, 48h, 72h, or 7d
is_recurring	BOOLEAN	Default: FALSE	Should it repeat?
recurrence_type	VARCHAR(20)	-	hourly, daily, weekly
status	VARCHAR(20)	Default: pending	pending, running, completed, failed
created_at	TIMESTAMP	Default: Current Time	When scheduler was created
last_run	DATETIME	-	When it last ran
next_run	DATETIME	-	When it will run next

4.4 scheduler_history table

Keeps record of every time a scheduler ran with results.

Column	Type	Rules	What It Stores
id	INT	Primary Key	Unique ID
scheduler_id	INT	Foreign Key to schedulers	Which scheduler ran
run_at	TIMESTAMP	Default: Current Time	When it ran
total_stocks	INT	Default: 0	How many stocks were in watchlist
total_news_fetched	INT	Default: 0	Total articles fetched
status	VARCHAR(20)	Default: completed	completed or failed
details	JSON	-	Per-stock article counts

4.5 search_history table

Tracks all manual searches user has done.

Column	Type	Rules	What It Stores
id	INT	Primary Key	Unique ID
symbol	VARCHAR(50)	NOT NULL	Which stock was searched
company_name	VARCHAR(200)	-	Company name
articles_found	INT	Default: 0	How many articles were found
time_filter	VARCHAR(20)	Default: 24h	Which time filter was used
searched_at	TIMESTAMP	Default: Current Time, Index	When search was done

5. All API Endpoints

Base URL: `http://localhost:5000` (or port 8888)

All APIs return JSON data. Here is the complete list:

5.1 Stock APIs

GET /api/stocks/search

Purpose: Autocomplete - search stocks as user types

Parameter: q = search text (minimum 1 character)

Returns: List of matching stocks with symbol, name, exchange

Example: `GET /api/stocks/search?q=TCS`

Response: `[{symbol: 'TCS', name: 'Tata Consultancy Services Ltd.', exchange: 'NSE'}]`

GET /api/stocks/resolve

Purpose: Convert stock name/symbol to full info

Parameter: q = stock symbol or name

Returns: symbol, name, exchange, in_watchlist (true/false)

Example: `GET /api/stocks/resolve?q=RELIANCE`

5.2 News APIs

GET /api/news/fetch

Purpose: Fetch LIVE news from Google News for a stock

Parameters: q = stock symbol/name, time = 24h/48h/72h/7d

Returns: success, stock info, articles list, total count, time_filter

Note: Also saves articles to database and merges with previously stored news

Example: `GET /api/news/fetch?q=INFY&time=24h`

GET /api/news/stored

Purpose: Get already stored news from database (no live fetch)

Parameters: symbol, time, source, scheduler_id (all optional)

Returns: List of stored articles

Example: `GET /api/news/stored?symbol=TCS&scheduler_id=1`

GET /api/news/for-history**Purpose:** Get news that was fetched for a specific search history entry**Parameters:** symbol, searched_at (timestamp), time_filter**Returns:** Articles that match the search criteria**5.3 Watchlist APIs**

Method	URL	What It Does
GET	/api/watchlist	Get all stocks in user's watchlist
POST	/api/watchlist	Add a stock. Send JSON: {symbol, company_name, exchange}
DELETE	/api/watchlist/<symbol>	Remove a stock. Example: DELETE /api/watchlist/TCS
POST	/api/watchlist/import	Upload .txt/.csv file to bulk add stocks (form-data with 'file')
DELETE	/api/watchlist/clear	Remove ALL stocks from watchlist

5.4 Scheduler APIs

Method	URL	What It Does
GET	/api/schedulers	Get all scheduler jobs
POST	/api/schedulers	Create new scheduler. JSON: {name, schedule_time, time_filter}
DELETE	/api/schedulers/<id>	Delete a scheduler by ID
POST	/api/schedulers/<id>/run	Manually run a scheduler NOW (don't wait for scheduled time)
GET	/api/scheduler-history	Get history of all scheduler runs
GET	/api/scheduler-history/<id>/news	Get news articles from a specific history run

5.5 Search History APIs

Method	URL	What It Does
GET	/api/search-history	Get last 20 searches user has done
DELETE	/api/search-history	Clear all search history

6. How Things Work (Flowcharts)

This section explains the step-by-step flow of main features. Mermaid diagram code is provided in the separate StockPulse_Diagrams.md file.

6.1 News Fetching Flow

What happens when user clicks 'Fetch News':

DIAGRAM: News Fetch Flow

User Search → Resolve Stock → Build RSS URL → Parse Feed → Filter Relevant → Analyze Sentiment
→ Save to DB → Return Results

Step by Step:

1. User types stock name (like 'TCS') and clicks Fetch News button
2. System looks up full stock info from stocks.py (167 stocks database)
3. Builds Google News RSS URL:
news.google.com/rss/search?q=TCS+share+price+when:24h
4. feedparser library fetches and parses the RSS XML
5. is_relevant_to_stock() function removes articles that don't actually mention the stock
6. simple_sentiment() analyzes each headline for positive/negative/neutral keywords
7. Articles saved to MySQL database (duplicates rejected using URL hash)
8. Live articles merged with previously stored articles (from scheduler runs)
9. Search saved to search_history table
10. JSON response sent to frontend which shows news cards

6.2 Scheduler Flow

How automatic background news fetching works:

DIAGRAM: Scheduler Flow

APScheduler (every 30 sec) → Check Due Jobs → Get Watchlist → Fetch News (5 parallel) → Save to DB → Update History

Step by Step:

11. APScheduler background thread runs check_due_schedulers() every 30 seconds
12. Queries database: SELECT * FROM schedulers WHERE next_run <= NOW() AND status = pending
13. For each due scheduler, first sets status = 'running' (prevents double execution)
14. Gets all stocks from user's watchlist
15. Uses ThreadPoolExecutor with 5 workers to fetch news for multiple stocks at same time
16. All fetched articles saved to news table with scheduler_id set
17. Creates entry in scheduler_history with total_stocks, total_news_fetched, and details JSON
18. Scheduler status set to 'completed'
19. If scheduler is recurring, calculates and sets next_run time
20. Special feature: If scheduler stuck in 'running' for >15 minutes, auto-resets to 'pending'

6.3 Sentiment Analysis Flow

How each article gets tagged as positive, negative, or neutral:

DIAGRAM: Sentiment Flow

Get Title → Lowercase → Count Positive Words → Count Negative Words → Compare → Assign Tag

Positive Keywords (22 words):

rally, surge, gain, rise, jump, soar, boost, profit, growth, upgrade, buy, bullish, high, record, beat, strong, outperform, positive, advance, recover, upbeat, boom

Negative Keywords (22 words):

fall, drop, crash, decline, loss, sell, bearish, low, downgrade, weak, plunge, slip, tumble, slump, fear, risk, cut, warning, negative, miss, underperform, recession

Logic:

- If positive_count > negative_count → POSITIVE (green)
- If negative_count > positive_count → NEGATIVE (red)
- If counts are equal (including both zero) → NEUTRAL (blue)

7. Step-by-Step Data Flow

7.1 Complete User Search Flow

From typing in search box to seeing news cards:

#	Where	What Happens	Data
1	Browser (app.js)	User types in search box	Raw text like 'reliance'
2	Browser (app.js)	Debounced autocomplete request (200ms delay)	GET /api/stocks/search?q=reliance
3	Flask (app.py)	Search in STOCK_DATABASE dict	Returns matching stocks
4	Browser (app.js)	User clicks 'Fetch News'	GET /api/news/fetch?q=...&time=24h
5	Flask (app.py)	Resolve stock symbol	RELIANCE → Reliance Industries Ltd.
6	news_fetcher.py	Build Google News RSS URL	news.google.com/rss/search?q=...
7	feedparser	Fetch and parse RSS XML	List of entry objects
8	news_fetcher.py	Filter irrelevant articles	Only articles mentioning stock
9	news_fetcher.py	Analyze sentiment of each	positive/negative/neutral tag added
10	database.py	Save to MySQL (with dedup)	INSERT with url_hash check
11	database.py	Get stored articles too	SELECT from news WHERE symbol=...
12	Flask (app.py)	Merge live + stored, remove duplicates	Combined unique article list
13	Browser (app.js)	Render news cards + update stats	DOM updates, sentiment counts

7.2 Database Connection Flow

StockPulse uses connection pooling (10 connections) to handle multiple requests:

- Pool created at app startup (Database.init_pool())
- Each request gets a connection from pool (not creating new one every time)
- After query completes, connection returned to pool (not closed)
- Autocommit enabled - changes saved immediately
- Character set: utf8mb4 (supports all languages and emojis)

7.3 Watchlist Import Formats

The bulk import feature supports 3 file formats:

21. Format 1: Company Name (SYMBOL) → Example: Reliance Industries Ltd.
(RELIANCE)
22. Format 2: SYMBOL - Company Name → Example: TCS - Tata Consultancy Services
23. Format 3: Just SYMBOL → Example: INFY (system looks up the name)

Each line in the file is one stock. Duplicates are automatically skipped.

8. Setup Requirements

8.1 What You Need Installed

- Python 3.8 or newer
- MySQL 5.7+ or 8.0+
- pip (Python package manager)
- Internet connection (to fetch news from Google)

8.2 Configuration Settings

Edit config.py or set these environment variables:

Setting	Default	What It Does
DB_HOST	localhost	MySQL server address
DB_PORT	3306	MySQL port number
DB_USER	root	MySQL username
DB_PASSWORD	(you must set)	MySQL password
DB_NAME	stockpulse	Database name (will be created)
NEWS_FETCH_TIMEOUT	15	Seconds to wait for Google News response
MAX_NEWS_PER_STOCK	30	Maximum articles to fetch per stock
SCHEDULER_CHECK_INTERVAL	30	How often (seconds) to check for due schedulers

8.3 How to Run

Step 1: Create database in MySQL

```
mysql> CREATE DATABASE stockpulse;
```

Step 2: Install Python packages

```
pip install -r requirements.txt
```

Step 3: Set your database password in config.py

Step 4: Run the application

```
python app.py
```

Step 5: Open browser at <http://localhost:5000> or :8888

9. Project Files Explained

Here is what each file in the project does:

File	What It Does
app.py	Main Flask server. Has all API routes (/api/news/fetch, /api/watchlist, etc). Starts on port 5000 or 8888. Initializes database and scheduler on startup.
config.py	Configuration settings - database connection, timeouts, limits. Can be overridden with environment variables.
database.py	All MySQL operations - create tables, save news, get watchlist, manage schedulers. Uses connection pooling (10 connections).
news_fetcher.py	Fetches news from Google News RSS. Has sentiment analysis (simple_sentiment), relevance filtering (is_relevant_to_stock), source detection (25+ sources with icons/colors).
stocks.py	Database of 167 stocks (NSE, BSE, NASDAQ, NYSE). Has resolve_stock() for lookup and search_stocks() for autocomplete.
scheduler.py	Background job system using APScheduler. Checks every 30 seconds for due jobs. Uses 5 parallel threads for fetching.
requirements.txt	List of Python packages needed: flask, mysql-connector-python, feedparser, requests, apscheduler
index.html	Main HTML page - search box, news grid, side panel for watchlist/scheduler, modals for history.
app.js	Frontend JavaScript - handles search, renders news cards, manages watchlist UI, updates sentiment counts, toast notifications.
style.css	Custom CSS - glassmorphism effects, animations (slide-up, float, pulse-glow), responsive design, dark theme.

10. What Can Be Added Later

Ideas for future improvements:

10.1 Coming Soon (Easy to Add)

Feature	What It Would Do	Difficulty
FinBERT Sentiment	Replace keyword matching with AI model trained on financial text - much more accurate	Medium
Email Alerts	Send email when important news found for watchlist stocks	Easy
More Time Filters	Add 1 hour, 12 hours, 30 days options	Easy
Export to Excel	Download fetched news as .xlsx file	Easy

10.2 Medium Term (Needs More Work)

Feature	What It Would Do	Difficulty
Live Stock Prices	Show current price next to stock name (using Yahoo Finance API)	Medium
User Accounts	Login system so each user has their own watchlist	Medium
Sentiment Charts	Show graph of sentiment over time for a stock	Medium
WebSocket Updates	Push new articles to browser without refresh	Medium

10.3 Long Term (Big Features)

- Mobile App: iOS and Android apps with push notifications
- Trading Signals: AI suggestions based on news sentiment patterns
- Multi-language: Support Hindi, Tamil, and other languages
- Broker Integration: Connect with Zerodha/Groww for one-click trading
- News Summary: Use AI to summarize multiple articles into one paragraph

— *End of Documentation* —

StockPulse v1.0 | Real-Time Stock News Aggregator